

SE3082 PARALLEL COMPUTING

LAB 07

MPI Collective Communication

Laboratory Report

Student Name	Kasthuriarachchi KACV
Student ID	IT24103440
Date	16 September 2026
Repository	se3082_pc_lab07_IT24103440

1. Objectives

- Understand MPI collective communication operations.
- Use MPI_Bcast, MPI_Scatter, MPI_Gather, MPI_Reduce, MPI_Allreduce, and MPI_Scan.
- Progressively refactor a parallel array-sum program using different collective communication patterns.
- Compare correctness, memory usage characteristics, result availability, and observed execution times.

2. Problem Statement

Task: Sum an integer array of 1,000,000 elements containing values from 1 to 1,000,000 using MPI.

Expected result: 500,000,500,000

Baseline commands: compile using mpicc and execute using mpirun. The number of processes must evenly divide 1,000,000.

```
mpicc -o program program.c
mpirun -np 4 ./program
```

3. Execution Environment

- Remote environment: Amazon Linux 2023 on an EC2 instance.
- Compiler: GCC 11.5.0 through mpicc.
- MPI implementation: Open MPI 4.1.2.
- Build automation: GNU Make.
- The EC2 instance exposed fewer MPI slots than required for 4- and 8-process runs, so Open MPI oversubscription was enabled for the laboratory execution.
- All six programs compiled successfully without reported compiler errors.

```
export PATH=/usr/lib64/openmpi/bin:$PATH
make clean
make
export OMPI_MCA_rmaps_base_oversubscribe=1
```

```
[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$ mpicc --version
mpirun --version
gcc (GCC) 11.5.0 20240719 (Red Hat 11.5.0-5)
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

mpirun (Open MPI) 4.1.2

Report bugs to http://www.open-mpi.org/community/help/

[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$ mpicc --version
mpirun --version
gcc (GCC) 11.5.0 20240719 (Red Hat 11.5.0-5)
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

mpirun (Open MPI) 4.1.2

Report bugs to http://www.open-mpi.org/community/help/
[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$ echo 'export PATH=/usr/lib64/openmpi/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$ mpicc --version
mpirun --version
gcc (GCC) 11.5.0 20240719 (Red Hat 11.5.0-5)
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

mpirun (Open MPI) 4.1.2

Report bugs to http://www.open-mpi.org/community/help/
[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$ make clean
rm -f sum_bcast sum_scatter sum_gather sum_reduce sum_allreduce sum_scan *.o
[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$ make
mpicc -Wall -Wextra -std=c99 -o sum_bcast sum_bcast.c
mpicc -Wall -Wextra -std=c99 -o sum_scatter sum_scatter.c
mpicc -Wall -Wextra -std=c99 -o sum_gather sum_gather.c
mpicc -Wall -Wextra -std=c99 -o sum_reduce sum_reduce.c
mpicc -Wall -Wextra -std=c99 -o sum_allreduce sum_allreduce.c
mpicc -Wall -Wextra -std=c99 -o sum_scan sum_scan.c
[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$ ls
Exercise02 Exercise04 Exercise06 Exercise08 Readme.md sum_allreduce sum_bcast sum_gather sum_reduce sum_scan sum_scatter
Exercise03 Exercise05 Exercise07 Makefile resources sum_allreduce.c sum_bcast.c sum_gather.c sum_reduce.c sum_scan.c sum_scatter.c
[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$
```

Figure 1. MPI environment and successful compilation evidence

4.1 Exercise 1: Broadcast + Send/Recv

Source file: sum_bcast.c

Communication pattern: MPI_Bcast + MPI_Send/MPI_Recv

Root creates the full array and broadcasts all 1,000,000 integers to every process. Each rank sums only its assigned index range. Non-root ranks send their partial sums to rank 0, which receives and accumulates them manually.

Key observations

- Every process allocates the full array.
- Root performs an O(P) receive loop for partial sums.
- All ranks must participate in MPI_Bcast.

Total sum = 500000500000

Expected = 500000500000

Correct? = YES

```
[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$ mpirun --oversubscribe -np 4 ./sum_bcast
Root filled array with values 1 to 1000000
Rank 0: summed indices [0, 250000) => local_sum = 31250125000
Rank 1: summed indices [250000, 500000) => local_sum = 93750125000
Rank 2: summed indices [500000, 750000) => local_sum = 156250125000
Rank 3: summed indices [750000, 1000000) => local_sum = 218750125000

[Bcast] Total sum    = 500000500000
[Bcast] Expected    = 500000500000
[Bcast] Correct?    = YES
[Bcast] Time        = 0.0124 sec
[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$
```

Figure 2. Exercise 1 execution and correctness verification

4.2 Exercise 2: Replace Broadcast with Scatter

Source file: sum_scatter.c

Communication pattern: MPI_Scatter + MPI_Send/MPI_Recv

Only rank 0 allocates the complete array. MPI_Scatter distributes N/size integers to each process, and every process sums only its local chunk. The point-to-point result collection from Exercise 1 is retained.

Key observations

- Non-root ranks allocate only a local chunk.
- MPI_Scatter sends only the data needed by each process.
- The sendcount is N/size, not N.

Total sum = 500000500000

Expected = 500000500000

Correct? = YES

```
[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$ mpirun --oversubscribe -np 4 ./sum_scatter
Root filled array with values 1 to 1000000
Rank 3: local_sum = 218750125000
Rank 2: local_sum = 156250125000
Rank 0: local_sum = 31250125000
Rank 1: local_sum = 93750125000

[Scatter] Total sum = 500000500000
[Scatter] Expected = 500000500000
[Scatter] Correct? = YES
[Scatter] Time = 0.0084 sec
[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$ |
```

Figure 3. Exercise 2 execution and correctness verification

4.3 Exercise 3: Replace Send/Recv with Gather

Source file: sum_gather.c

Communication pattern: MPI_Scatter + MPI_Gather

MPI_Gather replaces the manual Send/Recv result-collection loop. Each rank contributes one local_sum value, root receives them in rank order, and root manually sums the gathered values.

Key observations

- Root allocates all_sums with one long long per rank.
- MPI_Gather collects values but does not perform the summation.
- The final result is available on root after the manual sum.

Total sum = 500000500000

Expected = 500000500000

Correct? = YES

```
[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$ mpirun --oversubscribe -np 4 ./sum_gather
Root filled array with values 1 to 1000000
Rank 1: local_sum = 93750125000
Rank 3: local_sum = 218750125000
Rank 0: local_sum = 31250125000
Rank 2: local_sum = 156250125000

[Gather] Total sum = 500000500000
[Gather] Expected = 500000500000
[Gather] Correct? = YES
[Gather] Time = 0.0050 sec
[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$ |
```

Figure 4. Exercise 3 execution and correctness verification

4.4 Exercise 4: Replace Gather with Reduce

Source file: sum_reduce.c

Communication pattern: MPI_Scatter + MPI_Reduce (MPI_SUM)

MPI_Reduce combines all local sums using MPI_SUM and places the final total directly on rank 0. This removes both the gathered array and the manual root summation loop.

Key observations

- No all_sums buffer is required.
- MPI_SUM performs the reduction.
- The reduced result is valid only on root.

Total sum = 500000500000

Expected = 500000500000

Correct? = YES

```
[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$ mpirun --oversubscribe -np 4 ./sum_reduce
Root filled array with values 1 to 1000000
Rank 1: local_sum = 93750125000
Rank 2: local_sum = 156250125000
Rank 3: local_sum = 218750125000
Rank 0: local_sum = 31250125000

[Reduce] Total sum = 500000500000
[Reduce] Expected = 500000500000
[Reduce] Correct? = YES
[Reduce] Time = 0.0059 sec
[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$ |
```

Figure 5. Exercise 4 execution and correctness verification

4.5 Exercise 5: Replace Reduce with Allreduce

Source file: sum_allreduce.c

Communication pattern: MPI_Scatter + MPI_Allreduce (MPI_SUM)

MPI_Allreduce performs the same global sum but makes the result available to every rank. Each process therefore reports its local sum, the global total, and its percentage contribution.

Key observations

- Every rank receives total_sum.
- No root parameter is used in MPI_Allreduce.
- Percentage contribution can be computed locally on every process.

Total sum = 500000500000

Expected = 500000500000

Correct? = YES

```
[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$ mpirun --oversubscribe -np 4 ./sum_allreduce
Root filled array with values 1 to 1000000
Rank 1: local_sum = 93750125000, total_sum = 500000500000, percentage = 18.75%
Rank 2: local_sum = 156250125000, total_sum = 500000500000, percentage = 31.25%
Rank 0: local_sum = 31250125000, total_sum = 500000500000, percentage = 6.25%

[Allreduce] Total sum = 500000500000
[Allreduce] Expected = 500000500000
[Allreduce] Correct? = YES
[Allreduce] Time = 0.0051 sec
Rank 3: local_sum = 218750125000, total_sum = 500000500000, percentage = 43.75%
[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$ |
```

Figure 6. Exercise 5 execution and correctness verification

4.6 Exercise 6: Prefix Sums with Scan

Source file: sum_scan.c

Communication pattern: MPI_Scatter + MPI_Scan (MPI_SUM)

MPI_Scan computes a prefix reduction. Each rank receives the cumulative sum from rank 0 through itself and calculates `sum_before_me = prefix_sum - local_sum`. The last rank obtains the complete array sum.

Key observations

- Each rank receives a different prefix_sum.
- sum_before_me acts as a global offset.
- The final rank verifies that its prefix_sum is 500,000,500,000.

Observed 4-process prefix values:

Rank	Local sum	Prefix sum	Sum before me
0	31,250,125,000	31,250,125,000	0
1	93,750,125,000	125,000,250,000	31,250,125,000
2	156,250,125,000	281,250,375,000	125,000,250,000
3	218,750,125,000	500,000,500,000	281,250,375,000

Last rank prefix_sum = 500000500000

Expected = 500000500000

Correct? = YES

```
[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$ mpirun --oversubscribe -np 4 ./sum_scan
Root filled array with values 1 to 1000000
Rank 0: local_sum = 31250125000, prefix_sum = 31250125000, sum_before_me = 0
Rank 1: local_sum = 93750125000, prefix_sum = 125000250000, sum_before_me = 31250125000
Rank 2: local_sum = 156250125000, prefix_sum = 281250375000, sum_before_me = 125000250000
Rank 3: local_sum = 218750125000, prefix_sum = 500000500000, sum_before_me = 281250375000

[Scan] Total sum (last rank's prefix_sum) = 500000500000
[Scan] Expected = 500000500000
[Scan] Correct? = YES
[Scan] Time = 0.0138 sec
[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$
```

Figure 7. Exercise 6 execution and correctness verification

5. Exercise 7: Summary Comparison

5.1 Functional Comparison

Program	Collective / Communication	Full array on every rank?	Manual root sum?	Final result availability
sum_bcast	MPI_Bcast + Send/Recv	Yes	Yes	Root only
sum_scatter	MPI_Scatter + Send/Recv	No	Yes	Root only
sum_gather	MPI_Scatter + MPI_Gather	No	Yes	Root only
sum_reduce	MPI_Scatter + MPI_Reduce	No	No	Root only
sum_allreduce	MPI_Scatter + MPI_Allreduce	No	No	All processes
sum_scan	MPI_Scatter + MPI_Scan	No	No	Different prefix per rank

Note: In scatter-based programs, rank 0 still owns the complete input array; non-root ranks allocate only their local chunks.

5.2 Execution-Time Results

Each implementation was executed with 2, 4, and 8 MPI processes. The measured times are shown below.

Implementation	2 Processes	4 Processes	8 Processes
----------------	-------------	-------------	-------------

Bcast + Send/Recv	0.0057 s	0.0130 s	0.0163 s
Scatter + Send/Recv	0.0061 s	0.0055 s	0.0054 s
Scatter + Gather	0.0065 s	0.0064 s	0.0056 s
Scatter + Reduce	0.0061 s	0.0073 s	0.0060 s
Scatter + Allreduce	0.0061 s	0.0051 s	0.0061 s
Scatter + Scan	0.0128 s	0.0142 s	0.0072 s

Observed fastest runs: Bcast + Send/Recv was fastest at 2 processes (0.0057 s), Scatter + Allreduce was fastest at 4 processes (0.0051 s), and Scatter + Send/Recv was fastest at 8 processes (0.0054 s).

Interpretation: The fastest method changed with process count in this environment. Increasing process count did not always reduce elapsed time because collective communication, process coordination, and oversubscription overhead can offset the benefit of smaller local workloads. These timings should therefore be treated as observations from this EC2 laboratory environment rather than universal performance rankings.

```
[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$ mpirun -np 2 ./sum_bcast
Root filled array with values 1 to 1000000
Rank 0: summed indices [0, 500000) => local_sum = 125000250000
Rank 1: summed indices [500000, 1000000) => local_sum = 375000250000

[Bcast] Total sum    = 500000500000
[Bcast] Expected    = 500000500000
[Bcast] Correct?    = YES
[Bcast] Time        = 0.0057 sec
[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$ |

[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$ mpirun -np 4 ./sum_bcast
Root filled array with values 1 to 1000000
Rank 0: summed indices [0, 250000) => local_sum = 31250125000
Rank 2: summed indices [500000, 750000) => local_sum = 156250125000
Rank 3: summed indices [750000, 1000000) => local_sum = 218750125000
Rank 1: summed indices [250000, 500000) => local_sum = 93750125000

[Bcast] Total sum    = 500000500000
[Bcast] Expected    = 500000500000
[Bcast] Correct?    = YES
[Bcast] Time        = 0.0130 sec
[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$ |

[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$ mpirun -np 8 ./sum_bcast
Root filled array with values 1 to 1000000
Rank 4: summed indices [500000, 625000) => local_sum = 70312562500
Rank 7: summed indices [875000, 1000000) => local_sum = 117187562500
Rank 3: summed indices [375000, 500000) => local_sum = 54687562500
Rank 2: summed indices [250000, 375000) => local_sum = 39062562500
Rank 6: summed indices [750000, 875000) => local_sum = 101562562500
Rank 5: summed indices [625000, 750000) => local_sum = 85937562500
Rank 0: summed indices [0, 125000) => local_sum = 7812562500
Rank 1: summed indices [125000, 250000) => local_sum = 23437562500

[Bcast] Total sum    = 500000500000
[Bcast] Expected    = 500000500000
[Bcast] Correct?    = YES
[Bcast] Time        = 0.0163 sec
[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$ |
```

Figure 8. Representative benchmark evidence for 2, 4, and 8 process runs

5.3 Thinking Question: When to Use MPI_Scan

MPI_Scan is preferable to MPI_Allreduce when each process needs a cumulative result up to its own rank rather than the same global result on every process. A concrete example is distributed global index assignment: each rank can use the cumulative total of preceding ranks to determine the global offset at

which its local records should begin. This same pattern is useful for globally correct prefix sums, cumulative distributions, and work-offset calculations.

6. Exercise 8: Makefile and Automated Execution

A Makefile was created to compile all six programs and provide a run target that executes every implementation with four MPI processes.

```
make clean
make
make run
```

Execution note: The EC2 host initially reported insufficient MPI slots for four processes. After enabling Open MPI oversubscription, the make run target successfully executed all six programs and each reported the correct result.

```
export OMPI_MCA_rmaps_base_oversubscribe=1
make run
```

```
[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$ export OMPI_MCA_rmaps_base_oversubscribe=1
make run
Running with 4 processes:
=== sum_bcast ===
Root filled array with values 1 to 1000000
Rank 0: summed indices [0, 250000) => local_sum = 31250125000
Rank 1: summed indices [250000, 500000) => local_sum = 93750125000
Rank 2: summed indices [500000, 750000) => local_sum = 156250125000
Rank 3: summed indices [750000, 1000000) => local_sum = 218750125000

[Bcast] Total sum = 500000500000
[Bcast] Expected = 500000500000
[Bcast] Correct? = YES
[Bcast] Time = 0.0129 sec

=== sum_scatter ===
Rank 3: local_sum = 218750125000
Root filled array with values 1 to 1000000
Rank 1: local_sum = 93750125000
Rank 0: local_sum = 31250125000

[Scatter] Total sum = 500000500000
[Scatter] Expected = 500000500000
[Scatter] Correct? = YES
[Scatter] Time = 0.0045 sec
Rank 2: local_sum = 156250125000

=== sum_gather ===
Root filled array with values 1 to 1000000
Rank 3: local_sum = 218750125000
Rank 1: local_sum = 93750125000
Rank 2: local_sum = 156250125000
Rank 0: local_sum = 31250125000

[Gather] Total sum = 500000500000
[Gather] Expected = 500000500000
[Gather] Correct? = YES
[Gather] Time = 0.0050 sec

=== sum_allreduce ===
Root filled array with values 1 to 1000000
Rank 0: local_sum = 31250125000, total_sum = 500000500000, percentage = 6.25%

[Allreduce] Total sum = 500000500000
[Allreduce] Expected = 500000500000
[Allreduce] Correct? = YES
[Allreduce] Time = 0.0046 sec
Rank 1: local_sum = 93750125000, total_sum = 500000500000, percentage = 18.75%
Rank 2: local_sum = 156250125000, total_sum = 500000500000, percentage = 31.25%
Rank 3: local_sum = 218750125000, total_sum = 500000500000, percentage = 43.75%

=== sum_scan ===
Root filled array with values 1 to 1000000
Rank 0: local_sum = 31250125000, prefix_sum = 31250125000, sum_before_me = 0
Rank 1: local_sum = 93750125000, prefix_sum = 125000250000, sum_before_me = 31250125000
Rank 2: local_sum = 156250125000, prefix_sum = 281250375000, sum_before_me = 125000250000
Rank 3: local_sum = 218750125000, prefix_sum = 500000500000, sum_before_me = 281250375000

[Scan] Total sum (last rank's prefix_sum) = 500000500000
[Scan] Expected = 500000500000
[Scan] Correct? = YES
[Scan] Time = 0.0048 sec
```

Figure 9. Makefile run target executing all six MPI programs

7. Repository Submission

The completed source files and Makefile are stored in the GitHub repository. The final repository should include the following deliverables:

- sum_bcast.c
- sum_scatter.c
- sum_gather.c
- sum_reduce.c
- sum_allreduce.c
- sum_scan.c
- Makefile

```

ec2-user@ip-172-31-25-75:~/ / x + v
Verifying      : git-core-2.50.1-1.amzn2023.0.1.x86_64                2/8
Verifying      : git-core-doc-2.50.1-1.amzn2023.0.1.noarch          3/8
Verifying      : perl-Error-1:0.17030-2.amzn2023.0.1.noarch         4/8
Verifying      : perl-File-Find-1.37-477.amzn2023.0.9.noarch        5/8
Verifying      : perl-Git-2.50.1-1.amzn2023.0.1.noarch              6/8
Verifying      : perl-TermReadKey-2.38-9.amzn2023.0.3.x86_64        7/8
Verifying      : perl-lib-0.65-477.amzn2023.0.9.x86_64              8/8

Installed:
git-2.50.1-1.amzn2023.0.1.x86_64      git-core-2.50.1-1.amzn2023.0.1.x86_64
git-core-doc-2.50.1-1.amzn2023.0.1.noarch  perl-Error-1:0.17030-2.amzn2023.0.1.noarch
perl-File-Find-1.37-477.amzn2023.0.9.noarch  perl-Git-2.50.1-1.amzn2023.0.1.noarch
perl-TermReadKey-2.38-9.amzn2023.0.3.x86_64  perl-lib-0.65-477.amzn2023.0.9.x86_64

Complete!
[ec2-user@ip-172-31-25-75 ~]$ git --version
git version 2.50.1
[ec2-user@ip-172-31-25-75 ~]$ git clone https://github.com/chamika-kasthuri09/se3082_pc_lab07_IT24103440.git
Cloning into 'se3082_pc_lab07_IT24103440'...
remote: Enumerating objects: 30, done.
remote: Counting objects: 100% (30/30), done.
remote: Compressing objects: 100% (16/16), done.
remote: Total 30 (delta 6), reused 30 (delta 6), pack-reused 0 (from 0)
Receiving objects: 100% (30/30), 1.10 MiB | 6.65 MiB/s, done.
Resolving deltas: 100% (6/6), done.
[ec2-user@ip-172-31-25-75 ~]$ cd se3082_pc_lab07_IT24103440
[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$ ls
Exercise02 Exercise04 Exercise06 Exercise08 Readme.md sum_allreduce.c sum_gather.c sum_scan.c
Exercise03 Exercise05 Exercise07 Makefile resources sum_bcast.c sum_reduce.c sum_scatter.c
[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$ |

[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$ ls
Exercise02 Exercise04 Exercise06 Exercise08 Readme.md sum_allreduce.c sum_gather.c sum_scan.c
Exercise03 Exercise05 Exercise07 Makefile resources sum_bcast.c sum_reduce.c sum_scatter.c
[ec2-user@ip-172-31-25-75 se3082_pc_lab07_IT24103440]$

```

Figure 10. GitHub repository clone and final project contents

8. Conclusion

Lab 07 demonstrated the main MPI collective communication operations by progressively refactoring the same parallel array-sum problem. MPI_Bcast established the baseline but replicated the full input array on every rank. MPI_Scatter reduced non-root memory requirements by distributing only local chunks. MPI_Gather simplified result collection, MPI_Reduce combined collection and summation, MPI_Allreduce made the final total available to every process, and MPI_Scan produced rank-specific cumulative prefix results. All six implementations produced the expected total of 500,000,500,000. The timing experiment also showed that the fastest observed implementation varied with process count, highlighting the effect of communication and runtime overhead on parallel performance.

Appendix A: Commands Used for Verification

```
# Compile
make clean
make

# Four-process correctness runs
mpirun --oversubscribe -np 4 ./sum_bcast
mpirun --oversubscribe -np 4 ./sum_scatter
mpirun --oversubscribe -np 4 ./sum_gather
mpirun --oversubscribe -np 4 ./sum_reduce
mpirun --oversubscribe -np 4 ./sum_allreduce
mpirun --oversubscribe -np 4 ./sum_scan

# Benchmark pattern
mpirun --oversubscribe -np 2 ./<program>
mpirun --oversubscribe -np 4 ./<program>
mpirun --oversubscribe -np 8 ./<program>
```

Report prepared according to the Lab 07 README requirements. Execution evidence from the Linux/EC2 environment is included in the relevant sections.